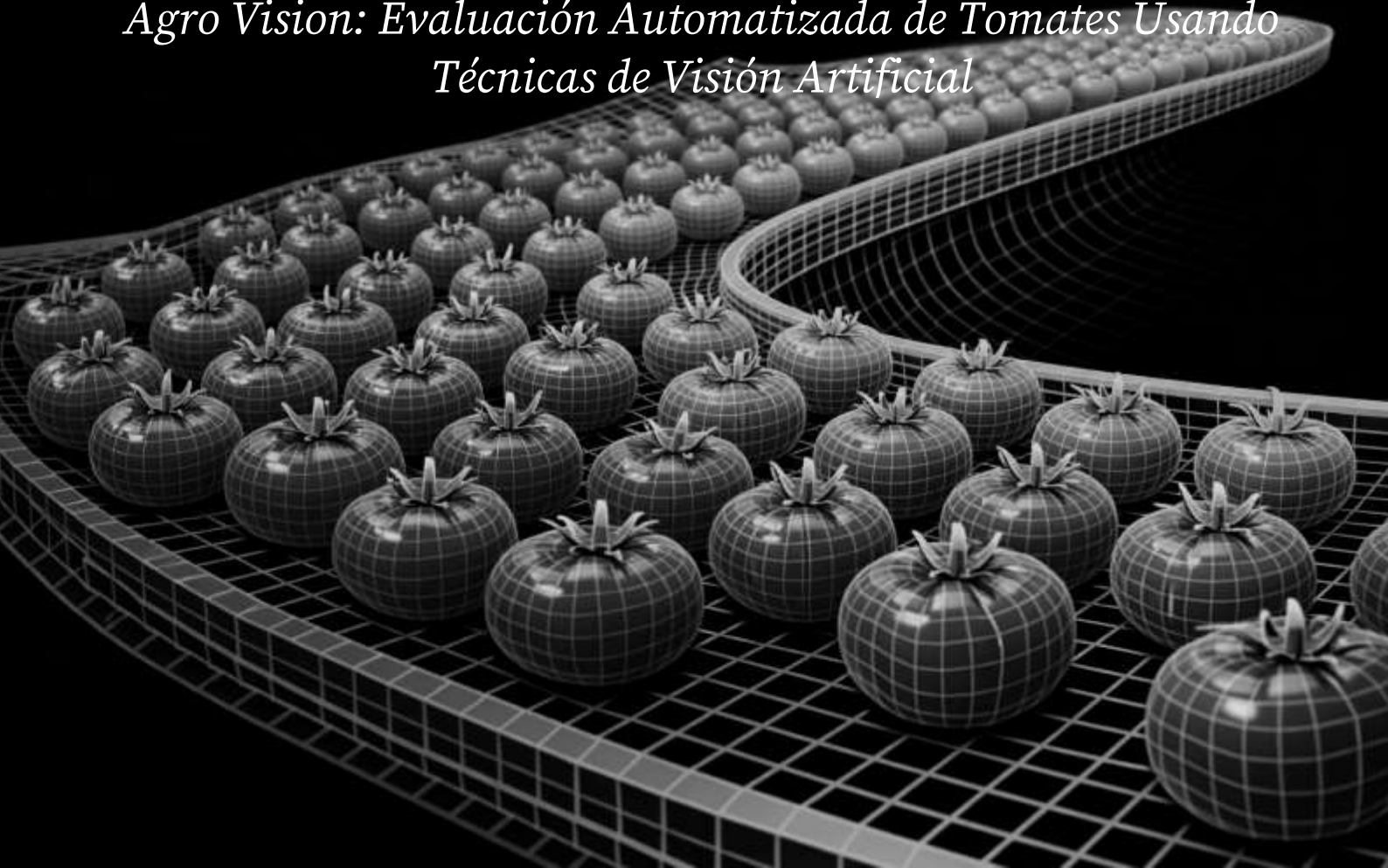




Universidad Autónoma del Estado de México

Manual de Usuario

Agro Vision: Evaluación Automatizada de Tomates Usando Técnicas de Visión Artificial



VISION ARTIFICIAL

INGENIRIA EN COMPUTACION

Autores:

- RAMIRO VEGA MEZA

Profesor:

Vianney Muñoz Jiménez

Versión 1.0 – 2026

INDICE

1. introducción	2
2. Objetivo	3
3. Requisitos del sistema	3
4. Instalación	4
5. Estructura del proyecto	5
6. Uso del sistema	6

1. introducción

El sistema AgroVision es una solución integral de visión por computadora orientada al análisis y procesamiento automático de productos agrícolas en tiempo real. Su propósito principal es la detección, clasificación y seguimiento de tomates utilizando técnicas modernas de inteligencia artificial, con el fin de optimizar procesos de inspección y selección en entornos agrícolas o industriales.

Este sistema emplea modelos de aprendizaje profundo basados en la arquitectura YOLO (You Only Look Once), una de las técnicas más eficientes para detección de objetos en tiempo real. A partir de la captura de video mediante dispositivos de entrada como cámaras web, AgroVision procesa cada cuadro para identificar objetos de interés (tomates) y clasificarlos automáticamente según su estado, como maduro, verde o defectuoso. Esta información se presenta al usuario mediante una interfaz visual que incluye cuadros delimitadores, etiquetas de clasificación y niveles de confianza.

Además, el sistema implementa el algoritmo de seguimiento SORT (Simple Online and Realtime Tracking), el cual permite mantener la identificación de cada objeto detectado a través de múltiples fotogramas, facilitando un análisis más consistente y robusto del flujo de video. La integración de esta técnica permite no solo detectar, sino

también seguir cada tomate dentro de la escena en tiempo real.

AgroVision está diseñado para ser flexible y escalable, pudiendo ejecutarse tanto en entornos con recursos limitados (CPU) como en sistemas con aceleración por hardware mediante GPU NVIDIA y tecnología CUDA. En este último caso, el rendimiento del sistema mejora significativamente, permitiendo alcanzar tasas de procesamiento más altas (FPS), lo cual es fundamental en aplicaciones en tiempo real.

El presente manual tiene como objetivo proporcionar al usuario una guía clara y detallada sobre la instalación, configuración y uso del sistema, así como ofrecer soluciones a problemas comunes que puedan surgir durante su ejecución. Está dirigido tanto a usuarios con conocimientos básicos de informática como a aquellos con experiencia en desarrollo, facilitando su implementación en distintos contextos.

En conjunto, AgroVision representa una herramienta tecnológica que aprovecha el potencial de la inteligencia artificial para mejorar la eficiencia, precisión y automatización en tareas relacionadas con la clasificación y monitoreo de productos agrícolas.

2. Objetivo

El objetivo principal del sistema **AgroVision** es desarrollar una herramienta automatizada de visión por computadora capaz de detectar, clasificar y realizar el seguimiento de tomates en tiempo real, con el fin de mejorar la eficiencia y precisión en procesos de inspección agrícola.

De manera específica, el sistema busca:

- Implementar un modelo de inteligencia artificial basado en **YOLO** para la detección precisa de tomates dentro de un flujo de video.
- Clasificar automáticamente los tomates según su estado (*maduro, verde o defectuoso*), facilitando la toma de decisiones en procesos de selección.
- Integrar técnicas de seguimiento mediante el algoritmo **SORT**, permitiendo identificar y mantener el registro de cada objeto detectado a lo largo del tiempo.
- Procesar video en tiempo real utilizando cámaras, optimizando el rendimiento mediante el uso de GPU cuando esté disponible.
- Reducir la intervención manual en tareas repetitivas, incrementando la productividad y disminuyendo el margen de error humano.

Adicionalmente, AgroVision tiene como objetivo ser un sistema flexible y accesible, que pueda ejecutarse tanto en equipos con recursos limitados (CPU) como en entornos con aceleración por hardware, adaptándose a diferentes

necesidades y contextos de implementación.

En conjunto, este sistema busca aportar una solución tecnológica eficiente para la automatización de procesos agrícolas, apoyándose en herramientas modernas de inteligencia artificial y visión computacional.

3. Requisitos del sistema

Para el correcto funcionamiento del sistema **AgroVision**, es necesario contar con los siguientes requisitos de hardware y software. Estos requisitos están divididos en mínimos y recomendados, dependiendo del nivel de rendimiento esperado.

Requisitos mínimos

- Procesador: Intel Core i3 o equivalente
- Memoria RAM: 4 GB
- Cámara: Webcam integrada o externa compatible con OpenCV
- Almacenamiento: 2 GB de espacio disponible

Requisitos recomendados

- Procesador: Intel Core i5 o superior
- Memoria RAM: 8 GB o más
- Tarjeta gráfica: GPU NVIDIA compatible con CUDA (ej. GTX 1050 o superior)
- Cámara: Cámara HD (720p o superior)

El uso de GPU permite mejorar significativamente el rendimiento del sistema, especialmente en tareas de detección en tiempo real.

Requisitos de software

- Sistema operativo: Windows 10 u 11
- Python: **versión 3.10**
- pip: actualizado a la última versión

Librerías necesarias

```
1 opencv-python
2 ultralytics
3 numpy
4 filterpy
5 scipy
6 torch
7 torchvision
8 torchaudio
```

Requisitos para aceleración por GPU (opcional)

Para aprovechar la aceleración mediante GPU:

- Tarjeta gráfica NVIDIA compatible
- Drivers NVIDIA instalados
- PyTorch compilado con soporte CUDA (ej. cu118)

Verificación de GPU:

```
1 import torch
2 print(torch.cuda.is_available())
```

4. Instalación

El usuario puede obtener el sistema de dos formas:

Opción 1: Descargar desde GitHub

```
git clone https://github.com/tu-usuario/AgroVis
cd AgroVisionProyecto
```

Opción 2: Carpeta manual

- Descargar el archivo del proyecto (.zip)
- Extraer el contenido
- Abrir la carpeta en la terminal

4.2 Crear entorno virtual

Se recomienda utilizar un entorno virtual para evitar conflictos entre librerías.

```
py -3.10 -m venv venv_gpu
```

Activar el entorno:

```
venv_gpu\Scripts\activate
```

Verificar versión:

```
python --version
```

4.3 Instalación de dependencias

Instalar las librerías necesarias para el funcionamiento del sistema:

```
pip install opencv-python ultralytics numpy filterpy scipy
```

4.4 Instalación de PyTorch con soporte GPU

Para aprovechar la aceleración por GPU (recomendado):

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
```

4.5 Verificación de instalación

Ejecutar el siguiente código para confirmar que la GPU está activa:

```
import torch
print(torch.cuda.is_available())
```

4.6 Verificación de archivos del proyecto

Asegúrese de que la estructura del proyecto contenga:

- Archivo principal: main.py
- Carpeta models/ con el modelo best.pt
- Archivos auxiliares (detector.py, tracker.py, etc.)

4.7 Ejecución del sistema

Una vez completados los pasos anteriores, ejecutar:

```
python main.py
```

4.8 Recomendaciones

- Mantener el entorno virtual activado durante la ejecución
- Verificar que la cámara esté conectada correctamente
- No modificar rutas internas del proyecto sin actualizar el código

5. Estructura del proyecto

El sistema AgroVision está organizado en módulos independientes que permiten separar claramente cada funcionalidad, facilitando su mantenimiento, comprensión y escalabilidad. A continuación, se describe la estructura general del proyecto y el propósito de cada archivo principal.



Descripción de archivos

main.py

Archivo principal del sistema.

- Controla el flujo general del programa
- Captura los frames de la cámara
- Ejecuta la detección y el tracking
- Muestra los resultados en pantalla
- Calcula y presenta los FPS

Es el punto de entrada del sistema.

camera.py

Módulo encargado de la captura de video.

- Inicializa la cámara
- Configura resolución y FPS
- Obtiene los frames en tiempo real

Permite aislar todo lo relacionado con la cámara.

detector.py

Encargado de la detección de objetos.

- Carga el modelo YOLO entrenado (best.pt)
- Ejecuta la inferencia sobre cada frame
- Devuelve las detecciones con:
 - Coordenadas
 - Clase (maduro, verde, defectuoso)
 - Confianza

Es el núcleo de inteligencia artificial del sistema.

tracker.py

Módulo de seguimiento de objetos.

- Utiliza el algoritmo SORT

- Asocia detecciones entre frames
- Asigna identificadores únicos a cada objeto

Permite seguir cada tomate en tiempo real.

sort.py

Implementación del algoritmo SORT.

- Usa filtros de Kalman (FilterPy)
- Gestiona la predicción y actualización de estados
- Optimiza el tracking en tiempo real

Funciona como base matemática del seguimiento.

train.py (opcional)

Script de entrenamiento del modelo.

- Permite entrenar YOLO con datos personalizados
- Configura épocas, tamaño de imagen y batch
- Genera el modelo best.pt

Solo se usa si se desea reentrenar el modelo.

Carpeta models/

Contiene el modelo entrenado:

models/

└─ best.pt

- Archivo esencial para la detección
- Generado después del entrenamiento
- Debe estar presente para que el sistema funcione

Archivos adicionales

requirements.txt

- Lista de dependencias del proyecto
- Permite instalar rápidamente todas las librerías

venv_gpu/

- Entorno virtual de Python
- Contiene las librerías instaladas

6. Uso del sistema

Esta sección describe cómo ejecutar y utilizar el sistema AgroVision una vez que ha sido instalado correctamente.

6.1 Inicio del sistema

Para iniciar el programa, asegúrese de que el entorno virtual esté activado:

```
venv_gpu\Scripts\activate
```

Luego, ejecutar:

```
python main.py
```

6.2 Funcionamiento general

Una vez iniciado el sistema:

1. Se activa la cámara automáticamente
2. Se comienza a capturar video en tiempo real
3. El modelo de inteligencia artificial analiza cada frame
4. Se detectan los tomates presentes en la imagen
5. Se clasifican según su estado:
 - Maduro
 - Verde
 - Defectuoso
6. Se aplica seguimiento a cada objeto detectado
7. Se muestran los resultados en pantalla

6.3 Visualización en pantalla

Durante la ejecución, el usuario observará:

-  Rectángulos (bounding boxes): delimitan cada tomate detectado
-  Etiqueta de clase: indica el estado del tomate
-  Confianza: probabilidad de acierto de la detección
-  Tracking ID: identificador único para cada objeto
-  FPS: número de cuadros procesados por segundo

6.4 Controles del sistema

- Control + C → Finaliza la ejecución del programa
- Cerrar la ventana → Detiene automáticamente el sistema

6.5 Comportamiento del sistema

- El sistema puede omitir detecciones en algunos frames para mejorar el rendimiento
- El tracking mantiene la identificación de los objetos incluso si no se detectan en todos los frames
- El rendimiento dependerá del hardware disponible (CPU o GPU)

